Universidad Abierta Interamericana
Facultad de Tecnología Informática
Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Diagramas del sistema

Paquetes, dominio, secuencia, estados, componentes, despliegue y casos de uso

Alumno: Pettini, Patricio Ezequiel
Legajo: B00072710-T1
Sede: Sede Norte — Boulogne
Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 23/10/2026

Índice

Los cincuenta que genera el script 3
1 · Paquetes y dependencias 4
2 · Modelo de dominio 5
3 · Secuencia · registrar una incidencia 6
4 · Estados de la incidencia 7
5 · Componentes en ejecución 8
6 · Despliegue 9
7 · Casos de uso por perfil 9
Fuentes 11

> *PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1*

Los siete diagramas salen de **Enterprise Architect**, que es la herramienta que recomienda la cátedra. No están dibujados a mano: los construye `docs/tecnico/_diagramas-ea.py` por la API de automatización de EA —paquetes, elementos, conectores y la posición de cada cosa— y le pide a EA que exporte la imagen.

Se automatiza por el mismo motivo que las capturas de pantalla: cuando el código cambia, los diagramas se rehacen con un comando. Un diagrama dibujado a mano se desactualiza y no hay forma de notarlo.

El modelo se entrega junto con las imágenes, en `docs/diagramas-ea/PredictIT.eapx`. Se abre en EA, se puede seguir editando a mano, y de ahí salen exactamente los PNG que están en este documento.

Cada sección dice de dónde sale el diagrama, para poder contrastarlo con el código en lugar de creerle.

Los cincuenta que genera el script

Este documento desarrolla **siete**, que son los que explican cómo está armado el sistema. El script genera cincuenta: el resto va al documento del Trabajo Final, en los apartados que los piden.

Tabla 1. *Inventario completo de diagramas*

Diagrama	Cuántos	Dónde
Paquetes, componentes, despliegue	3	Acá y en el TFI 10.4.5, 10.4.6 y 10.4.9
Casos de uso, vista general	1	Acá y en el TFI 10.4.2
Modelo de dominio, estados, secuencia	3	Acá
Caso de uso individual	18	TFI 10.4.3.1 a 10.4.3.18
Secuencia por caso de uso	16	TFI 10.4.4, uno por caso salvo los que no lo tienen
Clases de Bitácora	1	TFI 10.3.2.3.1
Clases de Usuario, Patente y Familia	1	TFI 10.3.2.5
Clases de la solución	3	TFI 10.4.7, partido en API+BLL, Service y DAO+Domain
Entidad-Relación	4	TFI 10.4.8

Los cuatro **DER no se dibujan**: se leen de las bases con `sys.tables` y `sys.foreign_key_columns`, así que muestran las tablas, las columnas, los tipos y las claves

foráneas que existen de verdad y no las que el diseño se propuso. Son 26 tablas y 46 foráneas en la base de negocio, y 15 y 16 en la de servicio.

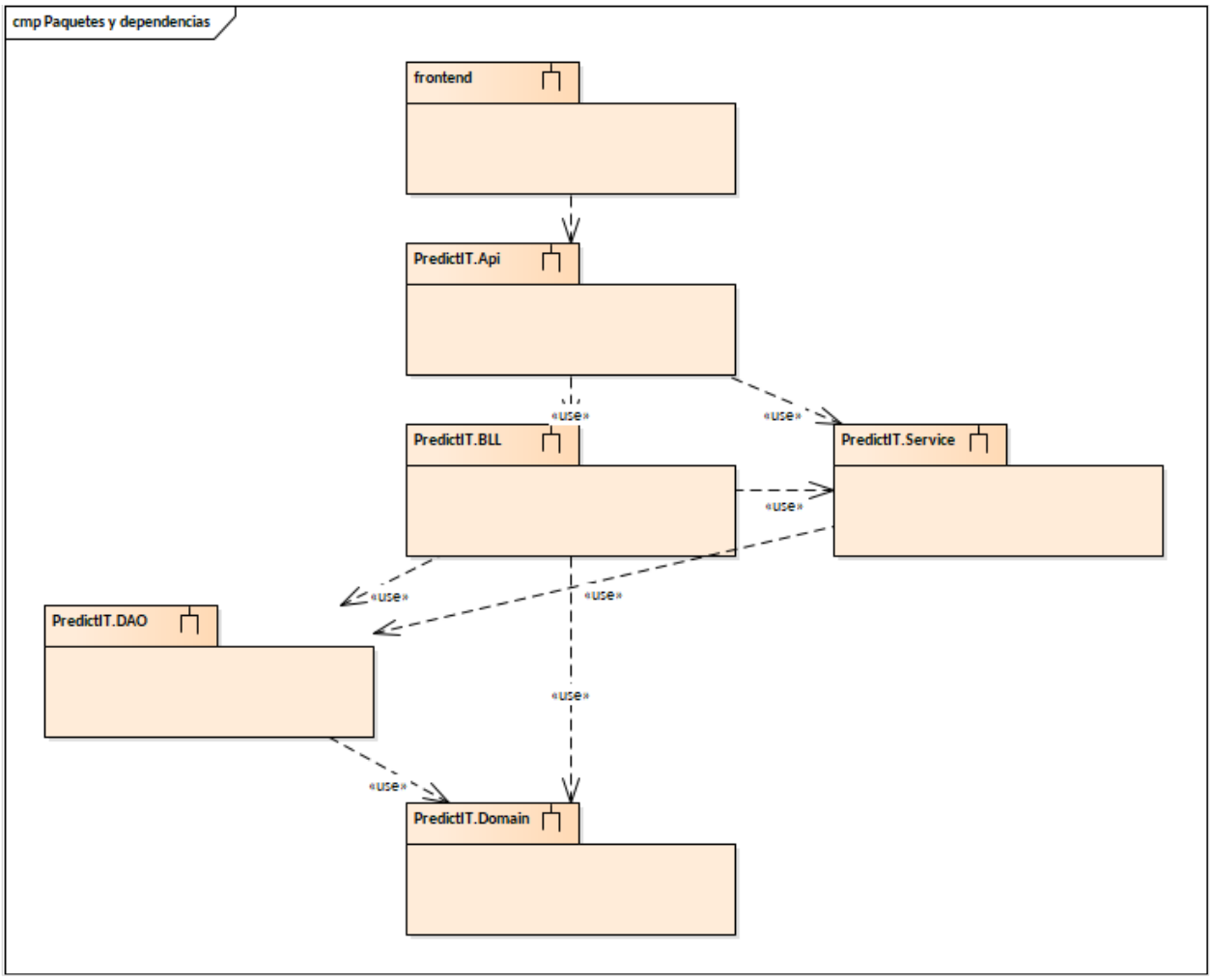
La base de negocio va partida en tres —equipos y mantenimiento, incidencias y asignación, y facturación— porque sus veintiséis tablas juntas entran en la hoja a un tercio de escala y el texto queda ilegible. Las tablas que quedan del otro lado de una relación se dibujan igual, con el nombre solo, para que la relación no desaparezca por donde se hizo el corte.

Las **dieciséis secuencias por caso de uso** salen de leer el controlador y el business de cada uno. A un diagrama de secuencia que no coincide con el código se le cree igual, y ahí está el daño.

1 · Paquetes y dependencias

Sale de las referencias declaradas en los `.csproj` y de `App.tsx`.

La regla que este diagrama tiene que hacer evidente: **las flechas van todas hacia abajo**. La API habla sólo con la BLL, el DAO es el único que conoce SQL, y el Domain no referencia a nadie.

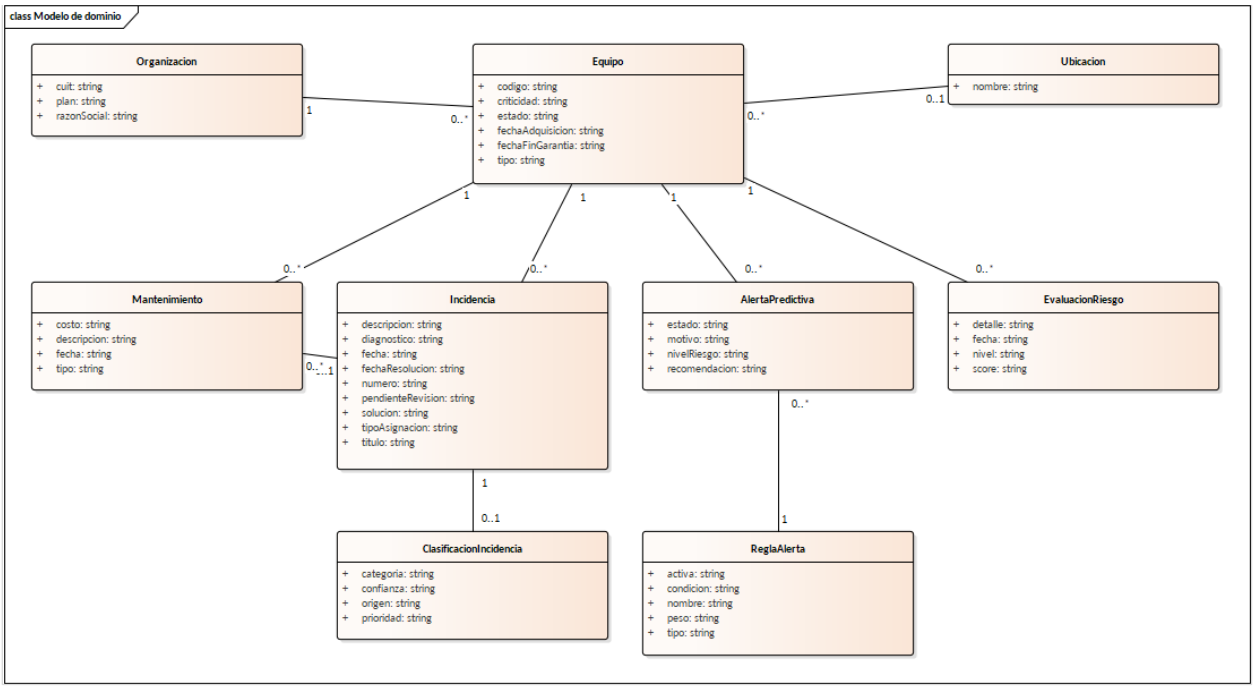


`Service` es transversal: lo consumen la BLL y la Api, y nunca al revés. Que dependa del DAO es lo que le permite a `BitacoraService` escribir sin pasar por el negocio, que es necesario porque tiene que poder asentar cuando el negocio falla.

2 · Modelo de dominio

Sale de las clases de `PredictIT.Domain` y de las claves foráneas de las dos bases (ver el diccionario de datos).

Se muestran las entidades y no los catálogos, que son treinta y siete tablas y harían el diagrama ilegible. La línea punteada entre las dos bases es una referencia lógica **sin clave foránea**: SQL Server no las admite entre bases distintas, y por eso los cruces se resuelven en memoria y por lote (ADR 0003).

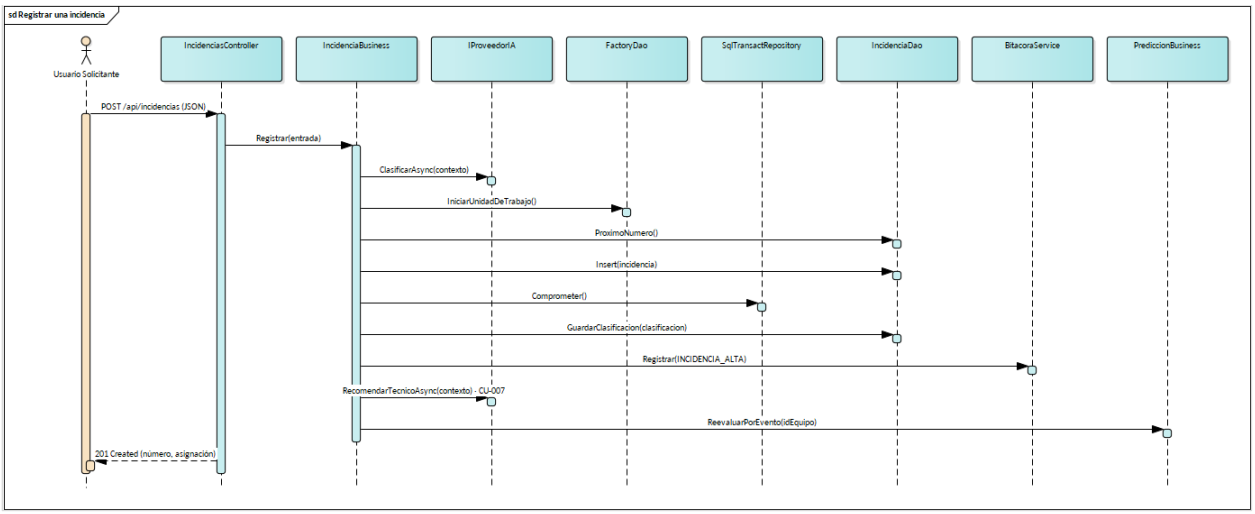


`FAMILIA ||--o{ FAMILIA` es el composite: una familia contiene familias. Es la relación que obliga a la guarda de ciclos, porque una familia que se contiene a sí misma cuelga la resolución de permisos.

3 · Secuencia · registrar una incidencia

Sale de `IncidenciaBusiness.Registrar`.

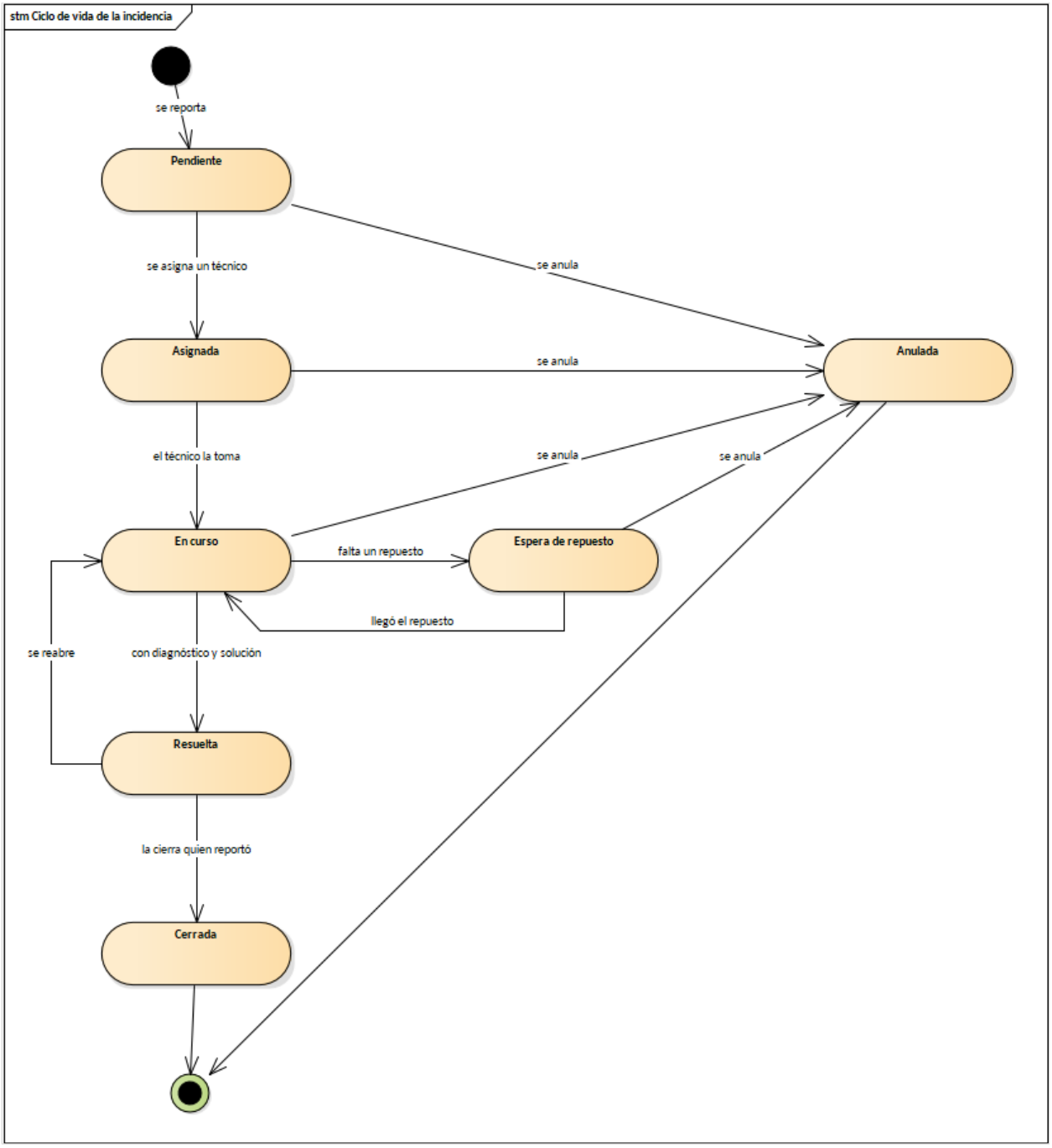
Es el flujo más denso del sistema y el que conviene poder explicar de memoria. Lo que el diagrama tiene que dejar claro: **los pasos 2 a 5 nunca hacen fallar el paso 1**. Que la asignación automática no funcione no puede impedir que se registre el reporte de una falla.



La asignación va **después** del alta y fuera de su transacción: si el proveedor tarda, no se sostiene una transacción abierta tomando bloqueos.

4 · Estados de la incidencia

Sale del diccionario `Permitidas` en `IncidenciaBusiness`.

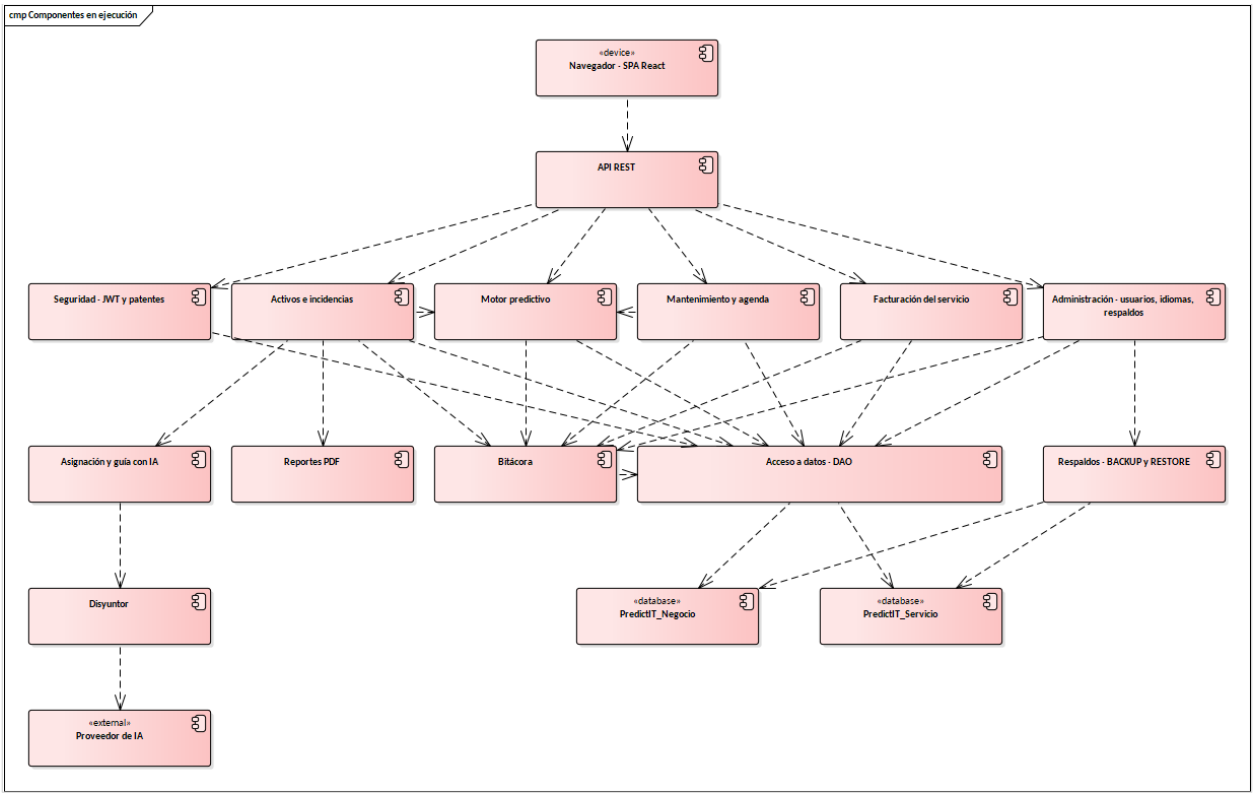


Dos cosas que el diagrama no puede mostrar y hay que decir:

- **Pasar a* Resuelta *exige diagnóstico y solución**, y sella la fecha de resolución.
- **La reapertura borra esa fecha.** El tiempo de reparación tiene que medirse hasta la resolución que quedó firme.

5 · Componentes en ejecución

Sale de `docker-compose.yml` y del registro de servicios en `Program.cs`.

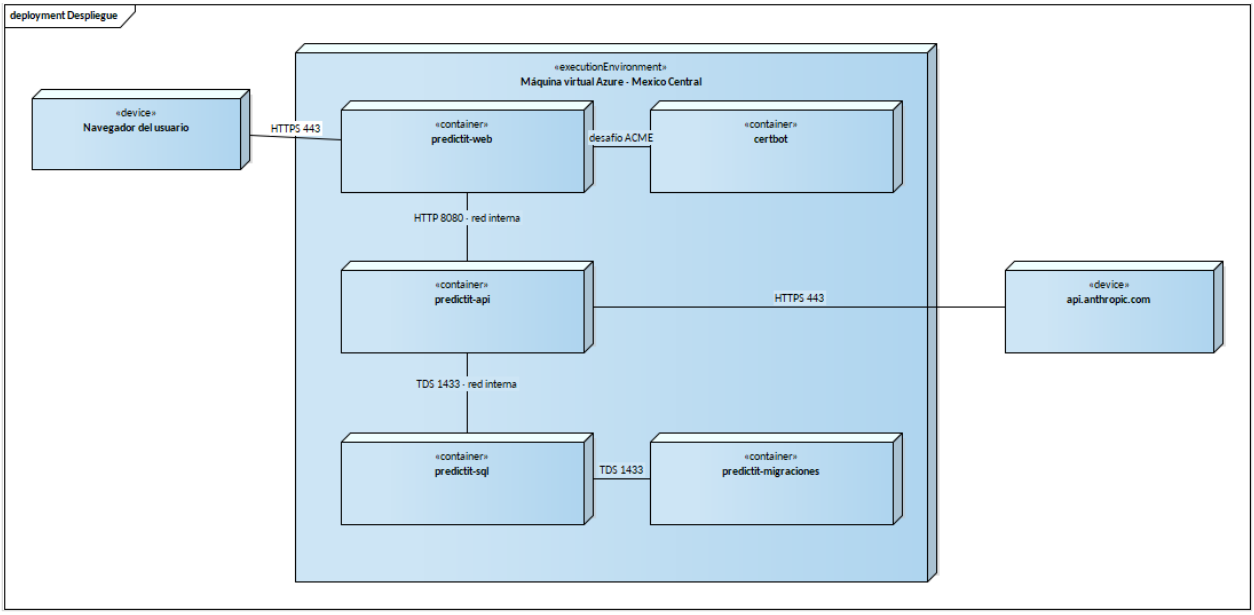


Tres cosas de este diagrama son decisiones y no accidentes:

- **El navegador nunca habla con la API directamente.** Pide a `/api` y nginx lo proxea, así la URL del backend no queda escrita en el bundle y no hay CORS que resolver.
- **El barrido usa un pool propio y acotado** (bulkhead, ADR 0009): puede agotar sus cinco conexiones y quedarse esperando las suyas, sin tocar las que atienden a los usuarios.
- **La flecha al servicio externo es punteada.** Sin clave de API el sistema usa el proveedor simulado y lo deja registrado en la bitácora: no deja de funcionar.

6 · Despliegue

Sale de `docker-compose.yml` y del manual de instalación.

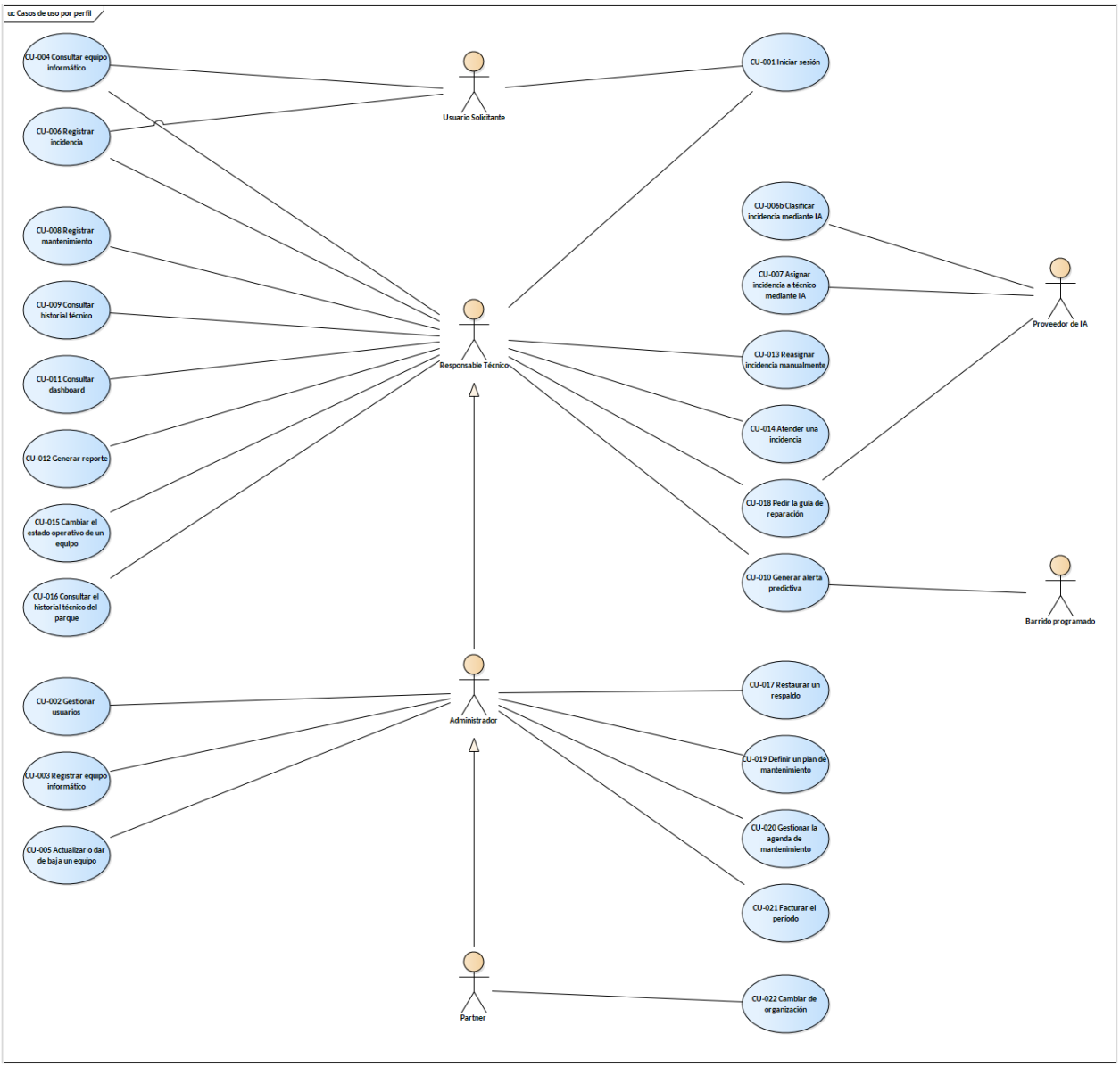


`predictit-migraciones` **corre una vez y termina**. La API espera a que termine bien: sin eso arrancaría contra una base sin tablas y el primer inicio de sesión fallaría con un error que no dice cuál es el problema real.

Los scripts son idempotentes, así que volver a levantar el stack reaplica y sale sin duplicar nada.

7 · Casos de uso por perfil

Sale de las patentes concedidas a cada familia en el seed.



El **Solicitante** ve sólo lo suyo, por decisión de diseño: si le mostráramos puntajes, pesos y prioridades, no reportaría nada. El **Partner** usa los casos del Administrador más el cambio de organización.

Se muestran los casos y no las veintiséis patentes. La correspondencia exacta entre caso de uso y patente está en el diccionario de datos y en el seed `db/03-seed-servicio.sql`.

_{Los diagramas se corrigen en el mismo commit que el cambio que los invalida. Si uno contradice al código, el código tiene razón y el diagrama es el defecto.}

Fuentes

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42-50.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Sparx Systems. (2024). *Enterprise Architect user guide*. https://sparxsystems.com/enterprise_architect_user_guide/